React basics

Your project introduction: VidyaSetu is a MERN course management website. React handles the interface, Express and Node.js handle API requests, and MongoDB stores users, courses, memberships, and uploaded notes.

These snippets are shortened from your project for revision. Study the behavior, not just the syntax. Main source: client/src/main.jsx.

1. Components and props

```
function Alert({ message }) {
  return message
    ? <div className="alert">{message}</div>
    : null;
}

<Alert message={error} />
```

A **component** is a reusable piece of the interface. **Props** pass data from a parent component to a child. Here, the component displays the message only when one exists.

Say in viva: *Alert is a reusable component. Its parent passes an error message through props.*

2. useState and click events

```
const [confirmLeave, setConfirmLeave] = useState(false);

<button onClick={() => setConfirmLeave(true)}>
  Leave course
</button>
```

confirmLeave stores the value. **setConfirmLeave** updates it. **false** is the initial value. Updating state makes React render the component again.

The arrow function runs when the button is clicked. Passing a function avoids calling the setter while rendering.

Say in viva: *I use state to manage course data, loading, errors, and confirmation messages.*

Inputs and loading data

3. Controlled input

```
const [query, setQuery] = useState('');

<input
  value={query}
  onChange={e => setQuery(e.target.value)}
/>
```

React state controls the input value. When the user types, **onChange** updates query. **e.target.value** is the current text inside the input.

Say in viva: *This is a controlled input because React state controls its value.*

4. useEffect and promises

```
useEffect(() => {
  if (!user) return;

  CourseService.list()
    .then(data => setCourses(data.courses))
    .catch(error => setError(error.message));
}, [user]);
```

This loads courses when the component mounts with a user, and again when user changes. **.then()** handles success; **.catch()** handles failure.

Say in viva: *I use useEffect to load courses from the backend.*

Dependency arrays:
[]: runs on mounting. Development StrictMode also performs an extra setup/cleanup cycle.
[user]: runs on mounting and when user changes.
No array: runs after every render.

Promise: an object representing a future result, such as an API response. It can succeed or fail.

Forms and API requests

5. Form submission with async/await

```
async function submit(e) {
  e.preventDefault();
  const data = Object.fromEntries(
    new FormData(e.currentTarget)
  );
  try {
    const result = await CourseService.enroll(data);
    navigate(`/courses/${result.course.id}`);
  } catch (error) {
    setError(error.message);
  }
}
```

preventDefault() stops normal form submission and page reload. **FormData** collects input values. **Object.fromEntries()** makes an object from those entries.

await waits for the result inside an async function without blocking the whole browser. **try/catch** handles errors. **navigate()** opens the course only after success.

Say in viva: *I collect form values, send them to the backend, and navigate after successful enrollment.*

6. fetch connects React to Express

```
const response = await fetch('/api/courses/enroll', {
  method: 'POST',
  credentials: 'same-origin',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(data)
});
const result = await response.json();
```

POST sends data. **JSON.stringify()** converts an object to JSON text. **response.json()** reads JSON as JavaScript data. **credentials** includes the session cookie for this origin.

Important: fetch does not automatically throw for HTTP errors such as 403 or 500. Your API helper checks response.ok. Source: client/src/api.js.

Lists and navigation

7. filter, map, and key

```
const files = materials.filter(material =>
  (material.category || 'notes') === category.value
);

{files.map(material => (
  <article key={material.id}>
    <h3>{material.title}</h3>
  </article>
))}
```

filter() selects files in the current category. **map()** turns each file into a UI element. **key** gives each item a stable identity so React can track changes. Missing categories default to Notes.

Say in viva: *Filter selects the required records, and map displays them.*

Source: client/src/CourseContent.jsx.

8. Routing and conditional rendering

```
<Route path="/courses/:id" element={
  user ? <CoursePage /> : <Navigate to="/login" />
} />

const { id } = useParams();
const navigate = useNavigate();

{canRemove && <RemoveControl />}
```

:id is a dynamic URL parameter. **useParams()** reads it. **useNavigate()** changes the route. The ternary operator chooses a page based on whether user exists. **&&** shows a control only when its condition is true.

Say in viva: *I show controls based on the role, but the backend also checks permissions for security.*

Remember the complete flow:
User action > React event handler > API request > Express validation > MongoDB > API response > React state update.

Quick viva answers

What is JSX?
JavaScript syntax for describing the interface with HTML-like markup.

Props vs state?
Props come from the parent; state is managed inside a component.

What is useRef used for here?
An immediate submission lock. Updating it does not cause a render.

Why useCallback here?
Keeps the course-loading function identity stable until its dependencies change.

What does user?.name mean?
Read name safely if user is null or undefined.

What does condition ? A : B mean?
Choose A when true; otherwise choose B.

Why use finally?
Clear loading and submission locks on both success and failure.

Why hash passwords on the backend?
Browser code can be inspected or bypassed. The server must verify passwords.

What is the service layer?
Reusable API functions such as CourseService.enroll and CourseService.leave.

What is a REST API?
HTTP endpoints that let the frontend work with backend resources.

What is the Virtual DOM?
React keeps an in-memory UI representation and reconciles changes before updating the DOM.

Why not mutate state directly?
Use the setter so React schedules the update and renders the new value.

Two security points to remember

1. Hiding an admin button in React is not authorization. Express checks the session and course role.
2. Enrollment passwords are stored as Argon2id hashes. The backend checks the password and assigns the member role.